

基于Docker本地化部署Coze：一份面向开发者的实践指南

从环境配置到服务启动，全流程保姆级解析



本次分享大纲：我们的“项目构建”蓝图

阶段一：奠定基石（环境准备）

- 系统要求与虚拟化技术开启
- Docker Desktop 的安装与核心配置



阶段二：组装核心（Coze配置）

- 获取Coze源码
- 配置大语言模型（LLM）API
- （可选）配置Embedding模型



阶段三：启动引擎（部署与验证）

- 使用 Docker Compose 启动服务
- 访问并验证本地Coze平台



阶段四：深度解析（总结与排错）

- 部署架构回顾
- 常见问题定位与解决方案



阶段一：环境准备 – 检查系统并开启虚拟化

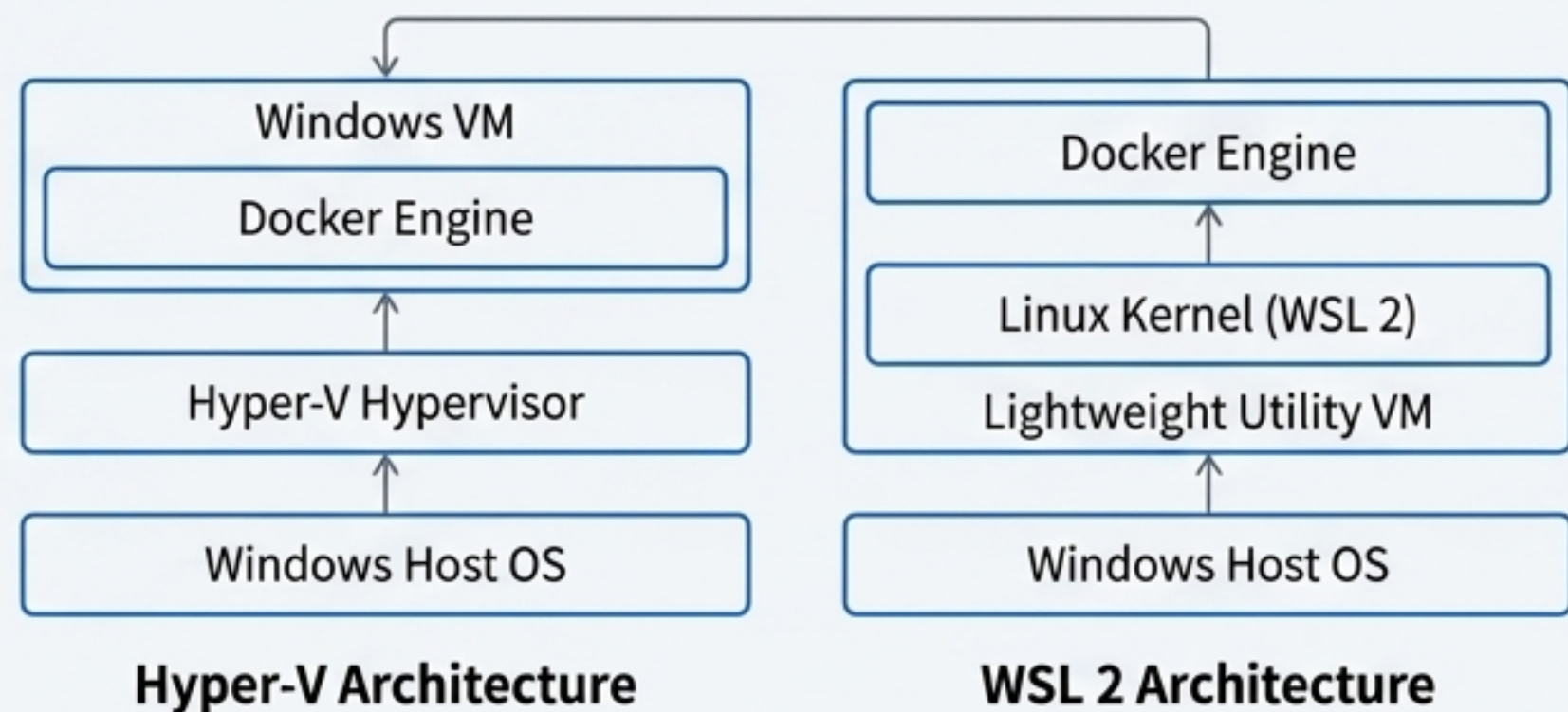
1. 部署前系统要求检查

- **操作系统：**Windows 10 64位（专业版、企业版或教育版，Build 19041 或更高）或 Windows 11 64位。
- **硬件：**至少 4GB RAM。

专家提示： 确保CPU虚拟化已在 BIOS/UEFI中启用。这是运行Docker Desktop的底层要求。

2. 开启Windows虚拟化功能（二选一）

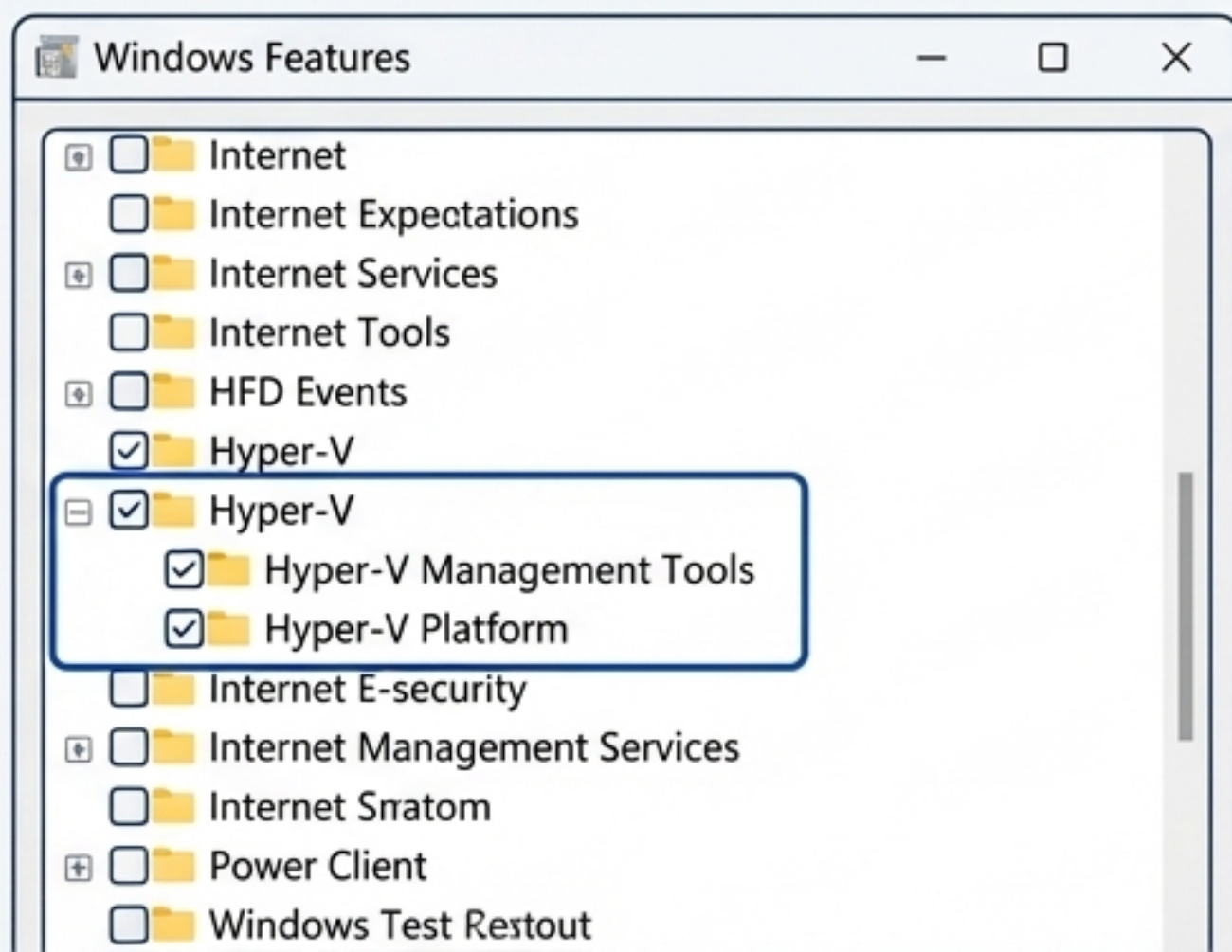
- **方案A：Hyper-V：**传统且稳定的虚拟化方案。
- **方案B：WSL（Windows Subsystem for Linux）：**官方推荐，性能更优，与Linux生态集成度更高。



实战：开启Hyper-V或安装WSL

方法一：通过UI启用Hyper-V

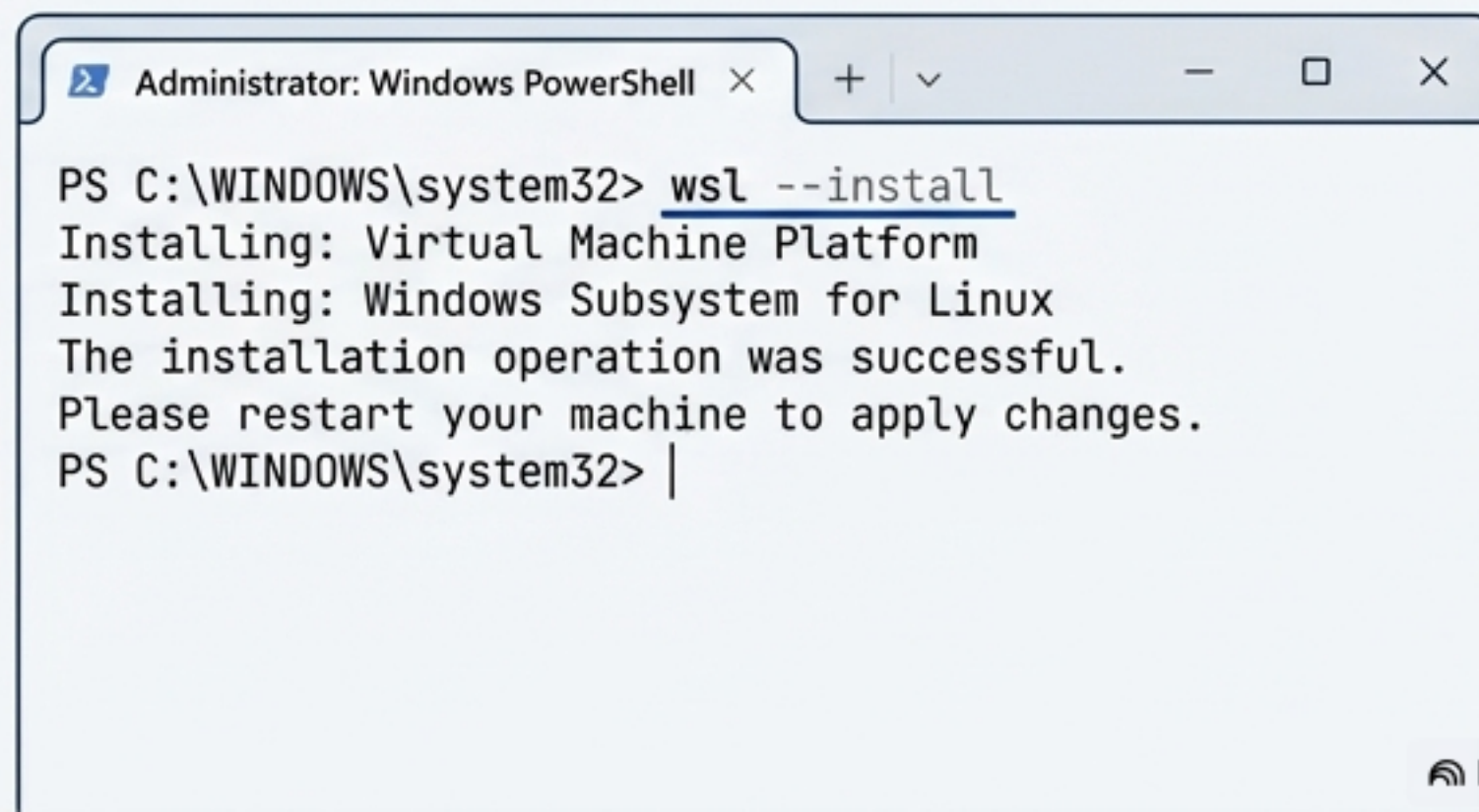
1. 打开“控制面板”->“程序”->“启用或关闭Windows功能”。
2. 在弹出的窗口中，勾选“Hyper-V”及其所有子选项。
3. 点击“确定”并重启计算机。



方法二：通过PowerShell安装WSL（推荐）

1. 以管理员身份运行 PowerShell。
2. 执行命令：

```
wsl --install
```
3. 安装完成后，建议执行 `wsl --set-default-version 2` 以确保使用WSL 2。



实战：安装Docker Desktop

1. 下载

- 访问Docker官方网站: ``https://docs.docker.com/get-docker/``
- 选择 'Docker Desktop for Windows'。

2. 安装（标准流程）

- 双击安装程序，按默认提示完成安装。

3. 专家提示：自定义安装到非系统盘（推荐）

- 为避免占用C盘空间，可通过命令行自定义安装路径。
- 以管理员身份打开终端，导航到安装包所在目录，执行：

```
start /w "" "Docker Desktop Installer.exe" install --installation-dir=D:\Program Files\Docker
```

4. 验证安装

- 安装完成后，在终端中输入 ``docker --version``，应能看到版本信息。

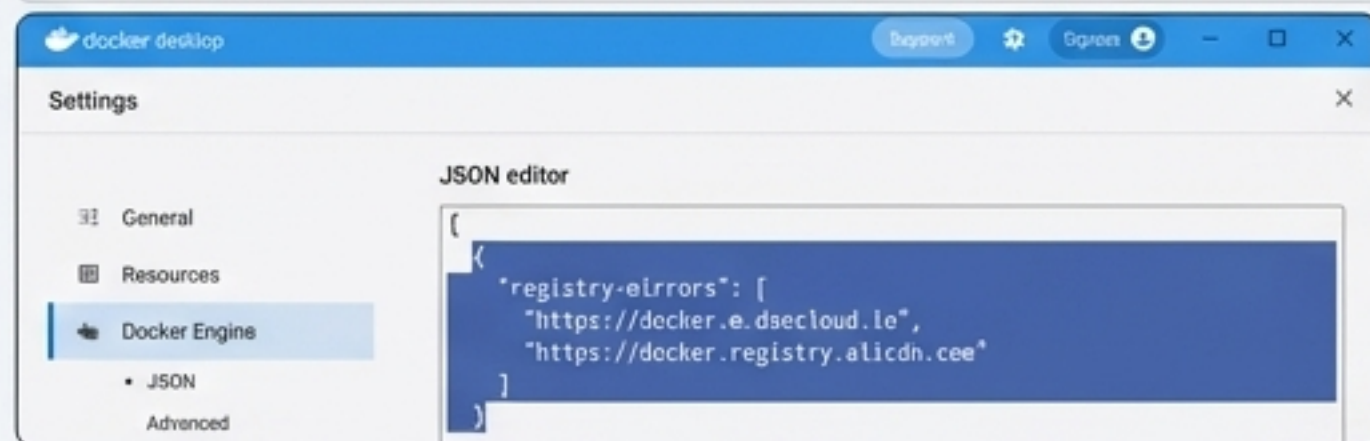


核心配置：优化Docker性能与资源管理

1. 配置国内镜像加速（关键步骤）

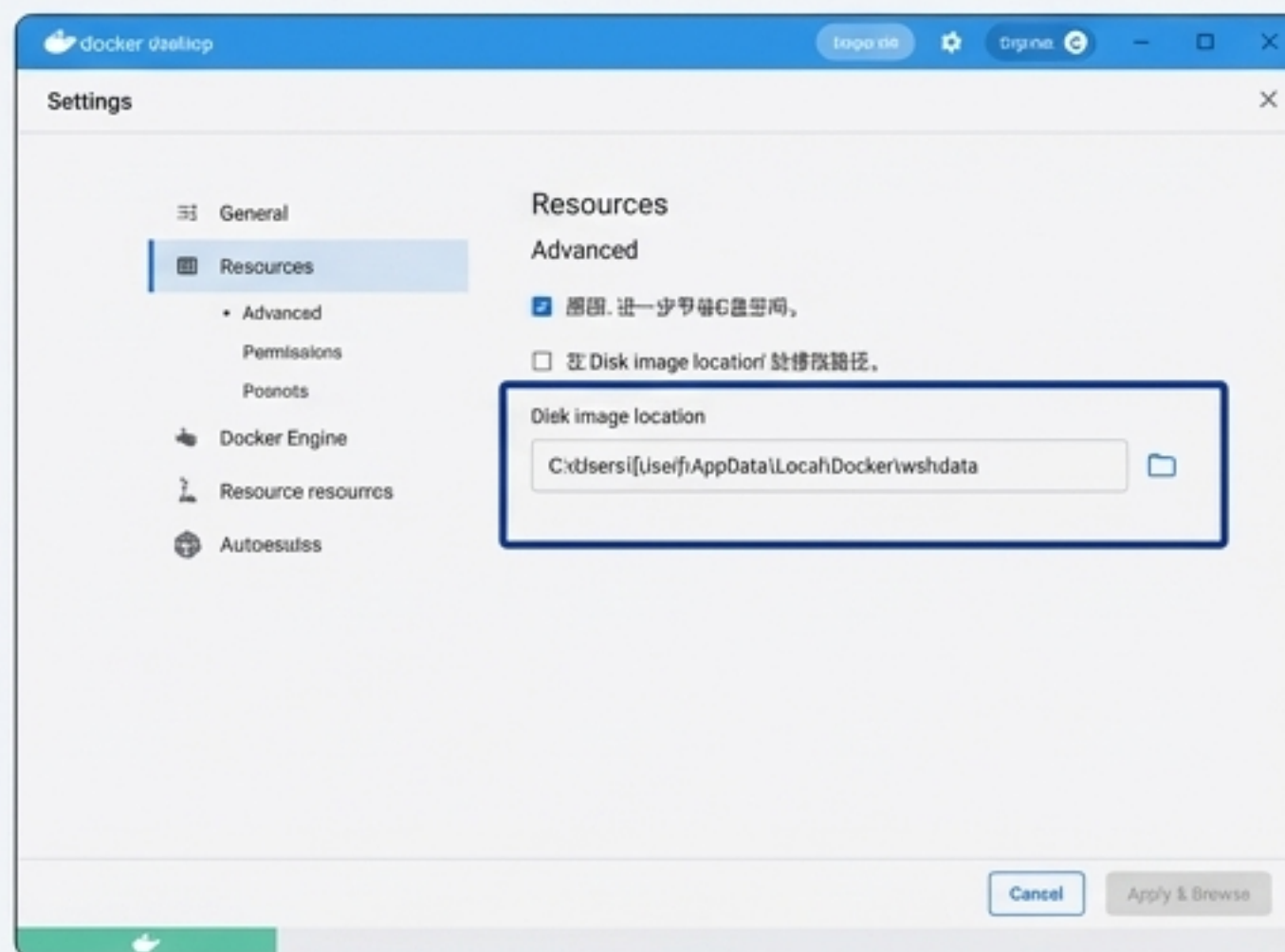
- **原因：**加速 `docker pull` 过程，避免因网络问题导致镜像拉取失败。
- **操作：**打开 Docker Desktop -> Settings -> Docker Engine。在JSON配置文件中添加 `registry-mirrors` 字段。
- **示例配置：**

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.registry.alicdn.com"
  ]
}
```



2. （可选）修改镜像和数据存储位置

- **原因：**进一步节省C盘空间。
- **操作：**Settings -> Resources -> Advanced。在“Disk image location”处修改路径。



阶段二：组装核心 - 获取Coze源码

准备工作：安装Git（推荐）

- Coze项目托管于GitHub，使用Git可以方便地获取更新。
- 下载地址：<https://git-scm.com/download/win>

获取源码（二选一）

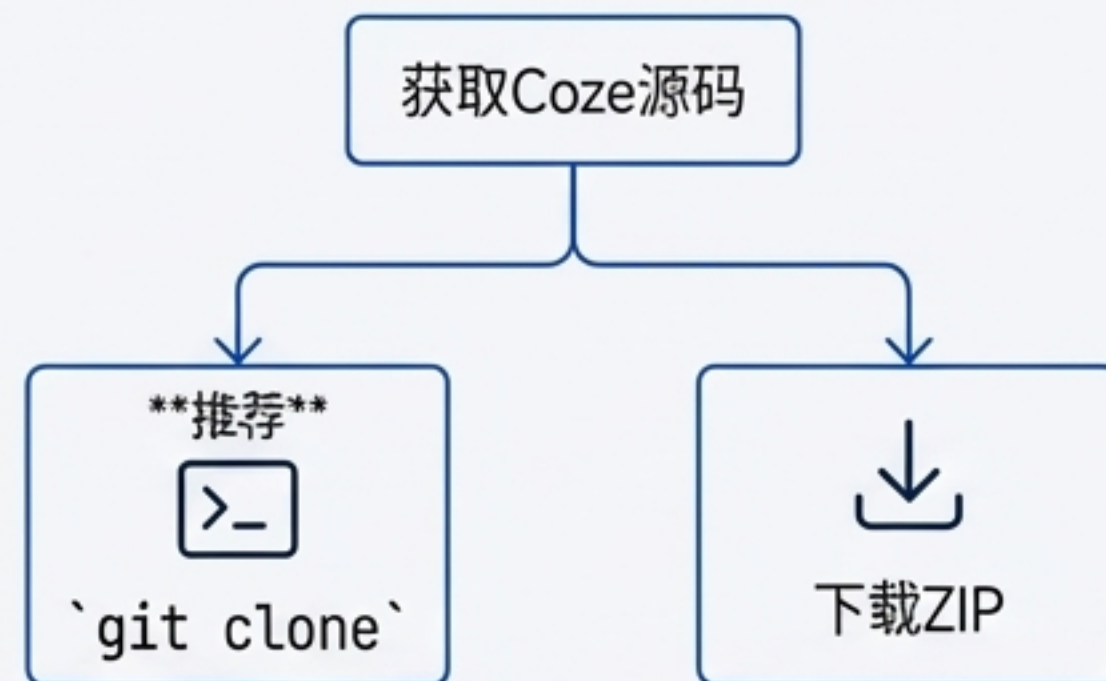
- 方法A（推荐）：`git clone`
- 在终端中执行以下命令，将项目克隆到本地：

```
git clone https://github.com/coze-dev/coze-studio.git
```

- ****方法B：下载ZIP压缩包****
- 直接从GitHub项目页面下载，但不利于后续版本更新。

验证工具版本

- 确保 `git --version`，`docker --version`，`docker compose version` 都能正常执行。

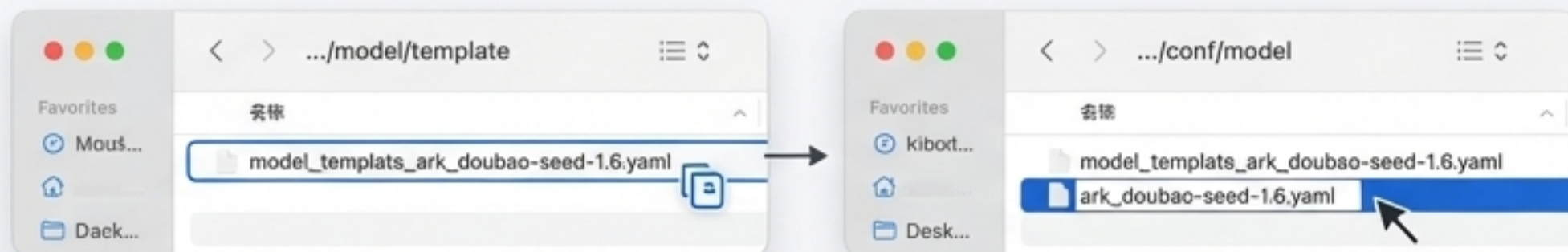


```
$ git clone https://github.com/coze-dev/coze-studio.git
Cloning into 'coze-studio'...
remote: Enumerating objects: 1234, done.
remote: Counting objects: 100% (1234/1234), done.
...
Resolving deltas: 100% (567/567), done.
```


关键步骤：配置大语言模型（LLM）连接

1. 定位并准备配置文件

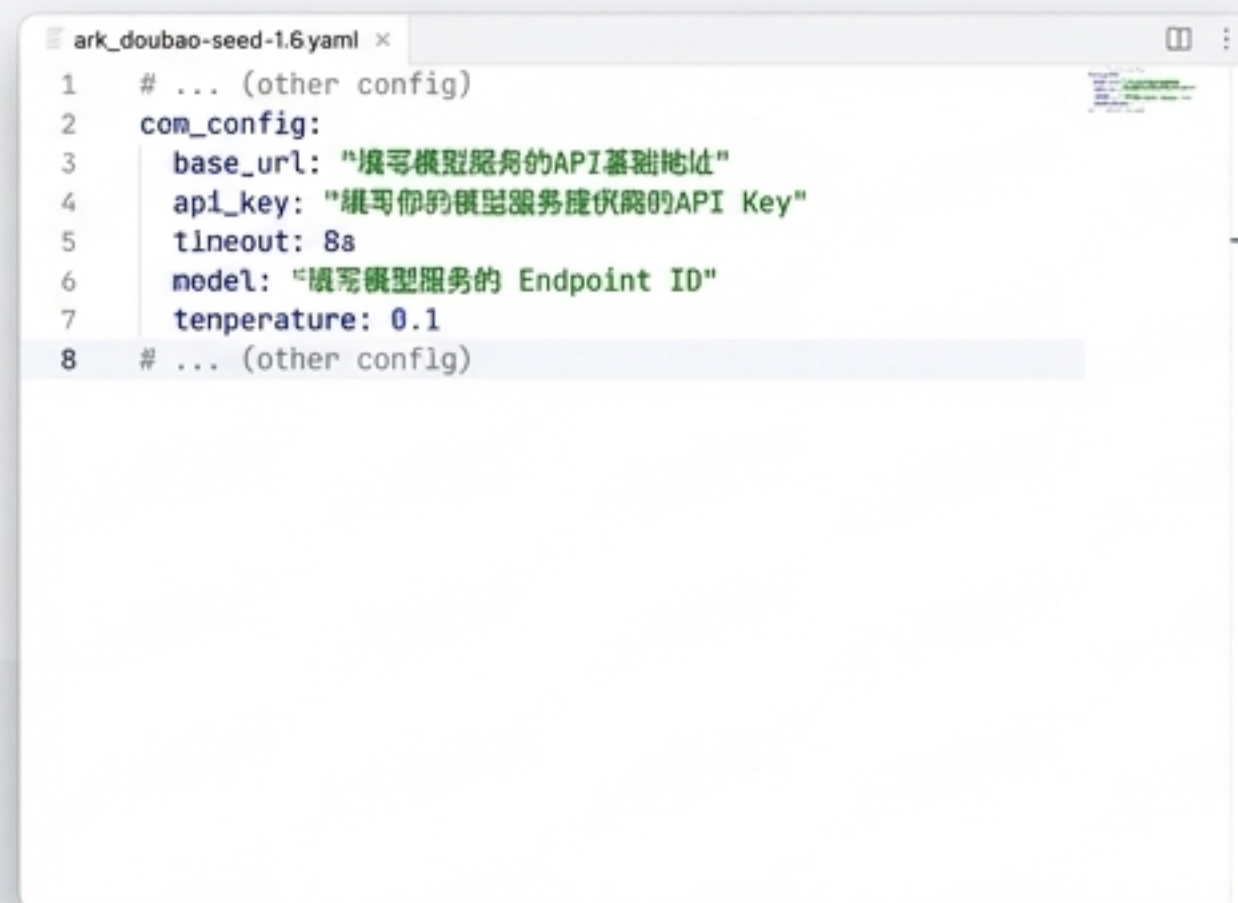
- 进入 `coze-studio/backend/conf/model/template` 目录。
- 复制 `model\_template\_ark\_doubao-seed-1.6.yaml` 文件。
- 将副本粘贴到上一级目录 `coze-studio/backend/conf/model` 中。
- 重命名为 `ark\_doubao-seed-1.6.yaml`。



2. 修改配置文件

- 用代码编辑器打开 `ark\_doubao-seed-1.6.yaml` 文件。
- **找到并修改 `com\_config` 部分的关键三项**：

```
# ... (other config)
com_config:
  base_url: "填写模型服务的API基础地址"
  api_key: "填写你的模型服务提供商的API Key"
  timeout: 8s
  model: "填写模型服务的 Endpoint ID"
  temperature: 0.1
# ... (other config)
```



实战：从火山方舟获取API密钥与模型Endpoint

1. 登录火山方舟平台

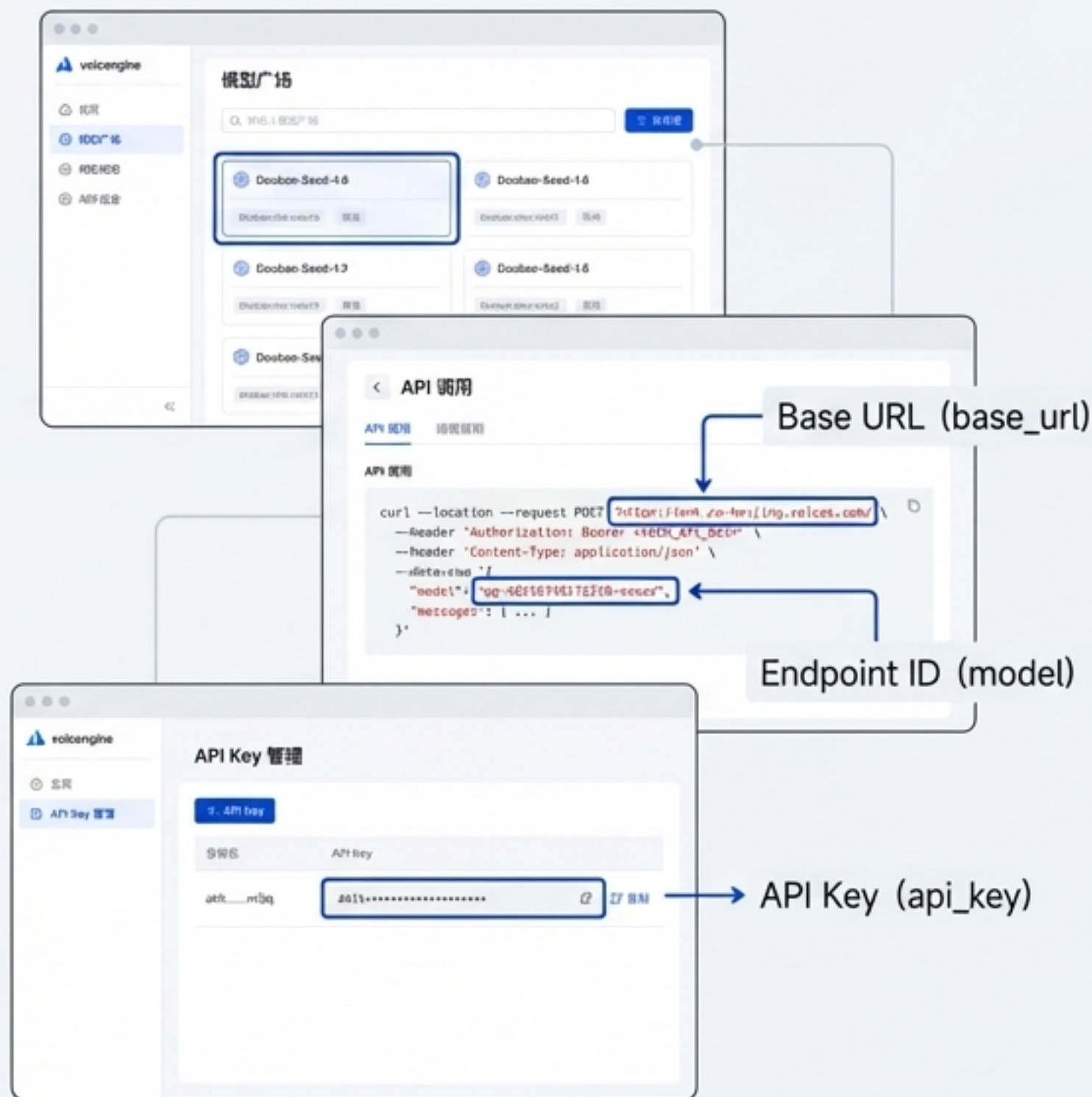
<https://www.volcengine.com/product/ark>

2. 创建模型服务

- 进入“模型广场”，选择并创建你需要的模型，例如 `Doubao-seed-1.6`。
- 火山方舟为新用户提供免费的Token额度。

3. 获取关键信息

- Endpoint ID (model)：在模型服务的“API调用”页面，Endpoint ID 即为 `model` 字段的值。
- Base URL (base_url)：REST API的请求地址中，域名部分即为 `base_url`。
- API Key (api_key)：在左侧导航栏的“API Key管理”页面创建或复制已有的Key。



进阶配置（可选）：添加Embedding模型

目的： 启用知识库、文档问答等需要向量化能力的AI功能。

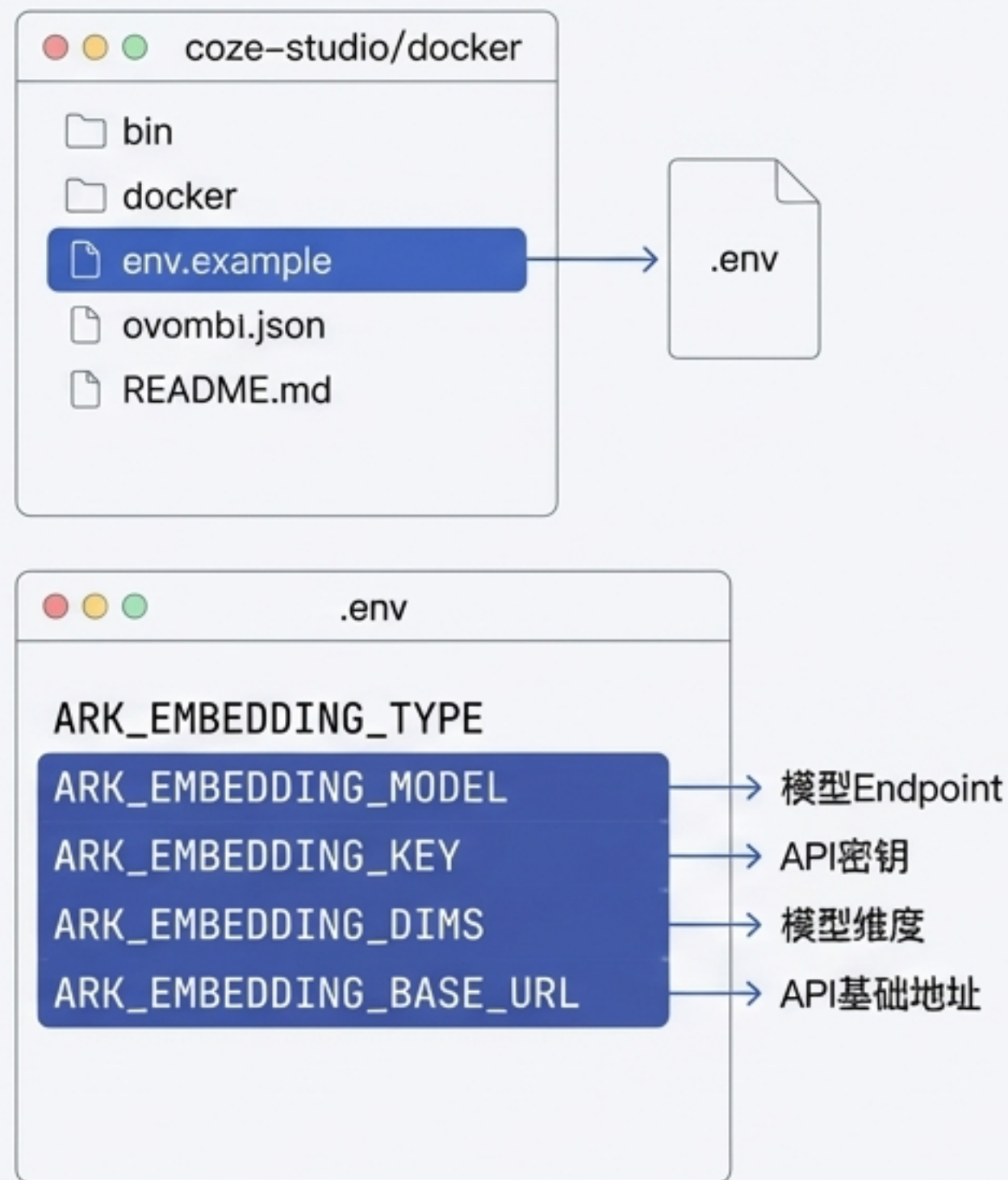
1. 准备环境变量文件

- 在 `coze-studio/docker` 目录下，复制 `env.example` 并重命名为 `.env`。

2. 配置Embedding参数

- 打开 `.env` 文件，填入从火山方舟获取的Embedding模型相关信息。
- 关键环境变量：

```
# ark embedding
export ARK_EMBEDDING_TYPE="ark"
export ARK_EMBEDDING_MODEL="<你的Embedding模型Endpoint ID>"
export ARK_EMBEDDING_KEY="<你的API Key>"
export ARK_EMBEDDING_DIMS="2048" # 根据模型填写
export ARK_EMBEDDING_BASE_URL="<你的Embedding模型API基础地址>"
```



阶段三：启动引擎 - 一键部署Coze服务

1. 进入部署目录

- 打开终端，确保当前路径位于 coze-studio/docker。

2. 执行启动命令

- 运行以下命令来构建并启动所有Coze服务容器：

```
docker compose --profile "*" up -d
```

- 命令解析：

- docker compose：使用compose工具来编排多容器应用。
- --profile "*"：激活在 docker-compose.yml 中定义的所有服务（包括基础服务和可选服务）。
- up：创建并启动容器。
- -d：（detached）在后台运行容器。

3. 观察启动过程

- 首次启动会下载所有必需的Docker镜像，耗时较长，请耐心等待。
- 终端会显示每个服务容器（如mysql, redis, coze-server等）的创建和启动状态。



```
Terminal
PS D:\projects\coze-studio\docker> docker compose --profile "*" up -d
[+] Running 10/10
 ✓ Network coze-network          Created
 ✓ Container coze-mysql          Started
 ✓ Container coze-redis          Started
 ✓ Container coze-server         Started
 ✓ Container coze-web            Started
 ✓ Container coze-backend        Started
 ✓ Container coze-gateway        Started
 ✓ Container coze-workFlow       Started
 ✓ Container coze-knowledge      Started
 ✓ Container coze-monitoring     Started
```


验证部署：访问你的本地Coze平台

访问地址

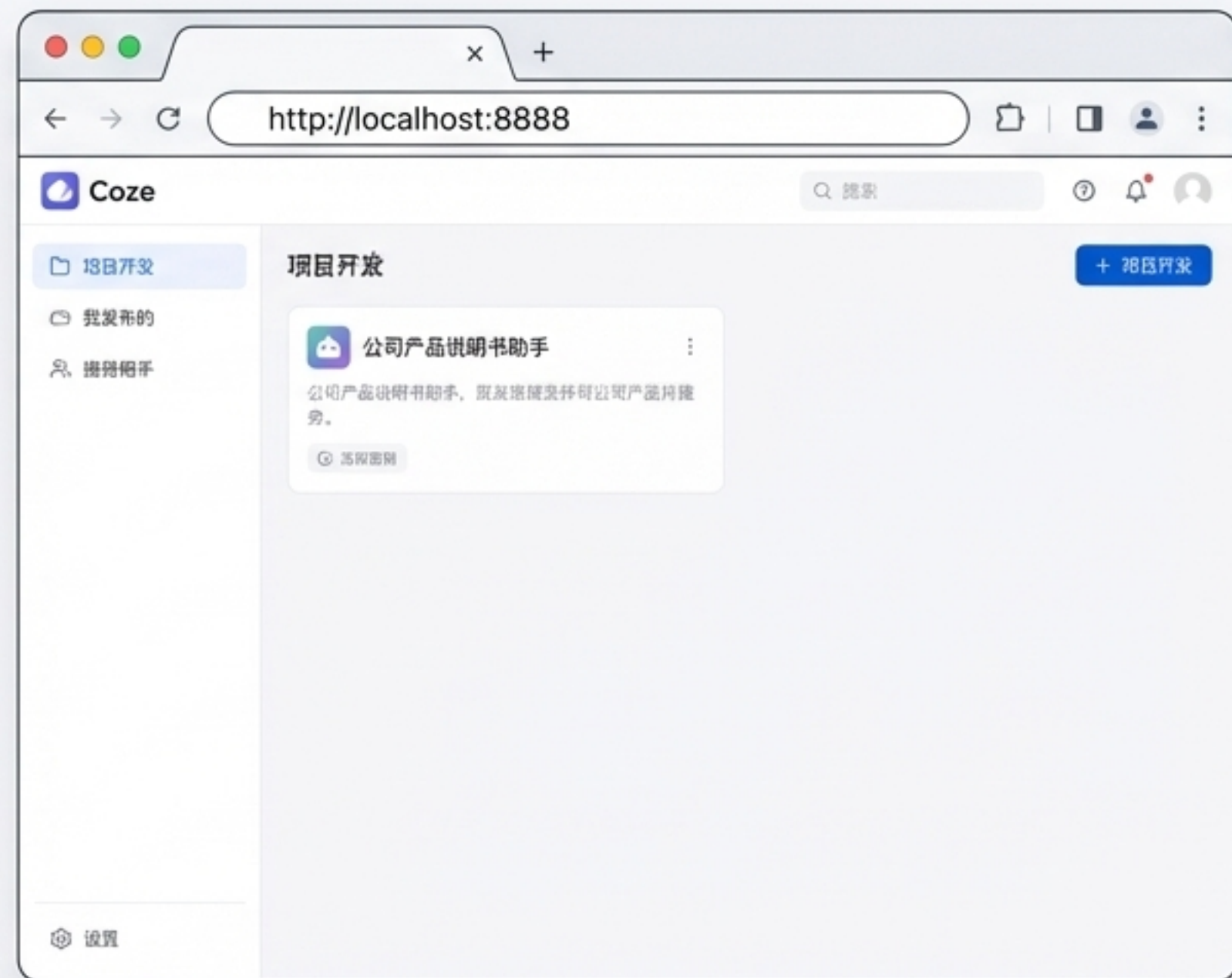
- 在浏览器中打开 `http://localhost:8888`

预期结果

- 你应该能看到Coze的欢迎界面或项目开发页面。
- 首次访问可能需要进行初始化设置。

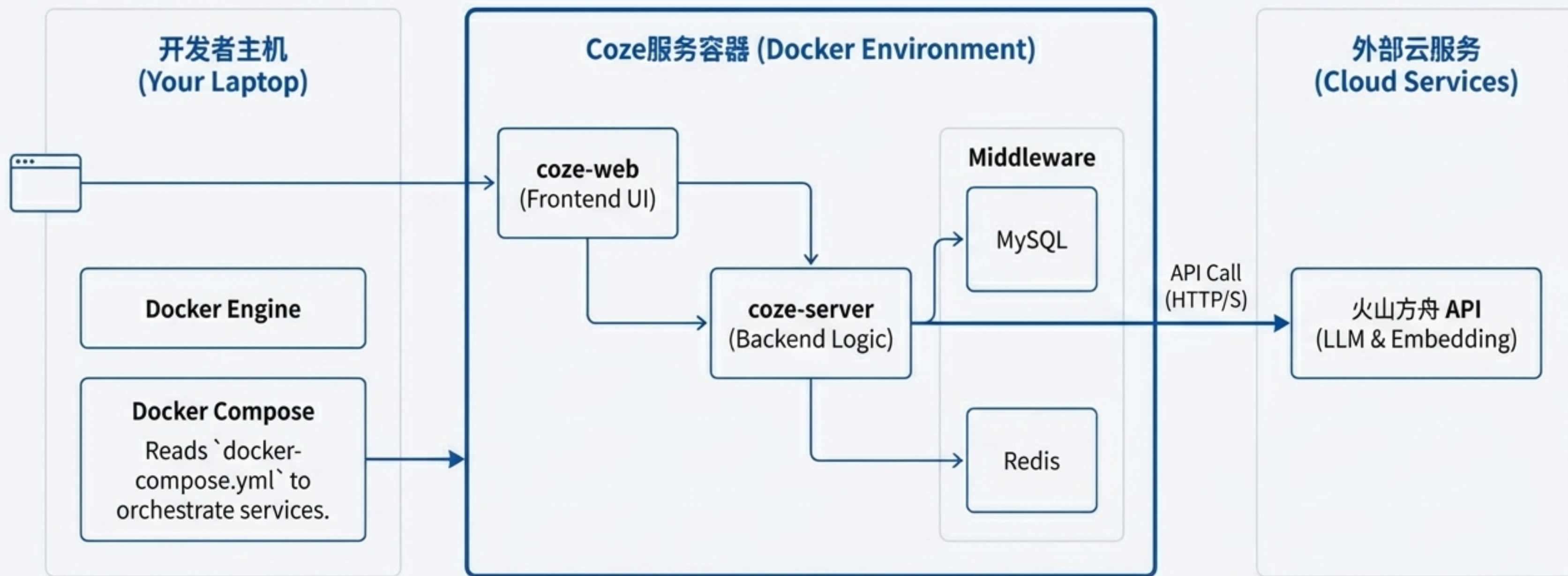
成功标志

- 能够正常加载页面，并看到“项目开发”、“个人空间”等UI元素，代表你的本地Coze实例已成功部署并运行。



阶段四：深度解析 – Coze本地部署架构

核心组件协同工作流程



常见问题排错指南 (Troubleshooting)

问题1: 端口冲突 (例如 3306)

- **现象:** `docker compose up` 报错, 提示 "port is already allocated" 或类似信息。
- **原因:** 你的主机上已经有其他服务 (如本地安装的MySQL) 占用了3306端口。
- **解决方案:**
 1. 打开 `coze-studio/docker/docker-compose.yml` 文件。
 2. 找到 `mysql` 服务的 `ports` 配置。
 3. 修改映射到主机的端口, 例如, 将 "3306:3306" 修改为 "3307:3306"。
 4. 修改后, Coze内部仍使用3306端口访问MySQL, 但它在主机上的映射端口变为3307, 避免了冲突。

BEFORE

```
services:
  mysql:
    #...
    ports:
      - "3306:3306"
```



AFTER

```
services:
  mysql:
    #...
    ports:
      - "3307:3306"
```

问题2: 镜像拉取缓慢或失败

- **原因:** 网络问题, 无法连接到Docker Hub。
- **解决方案:** 回到“核心配置”步骤, 检查并确保你已正确配置了国内镜像加速器。

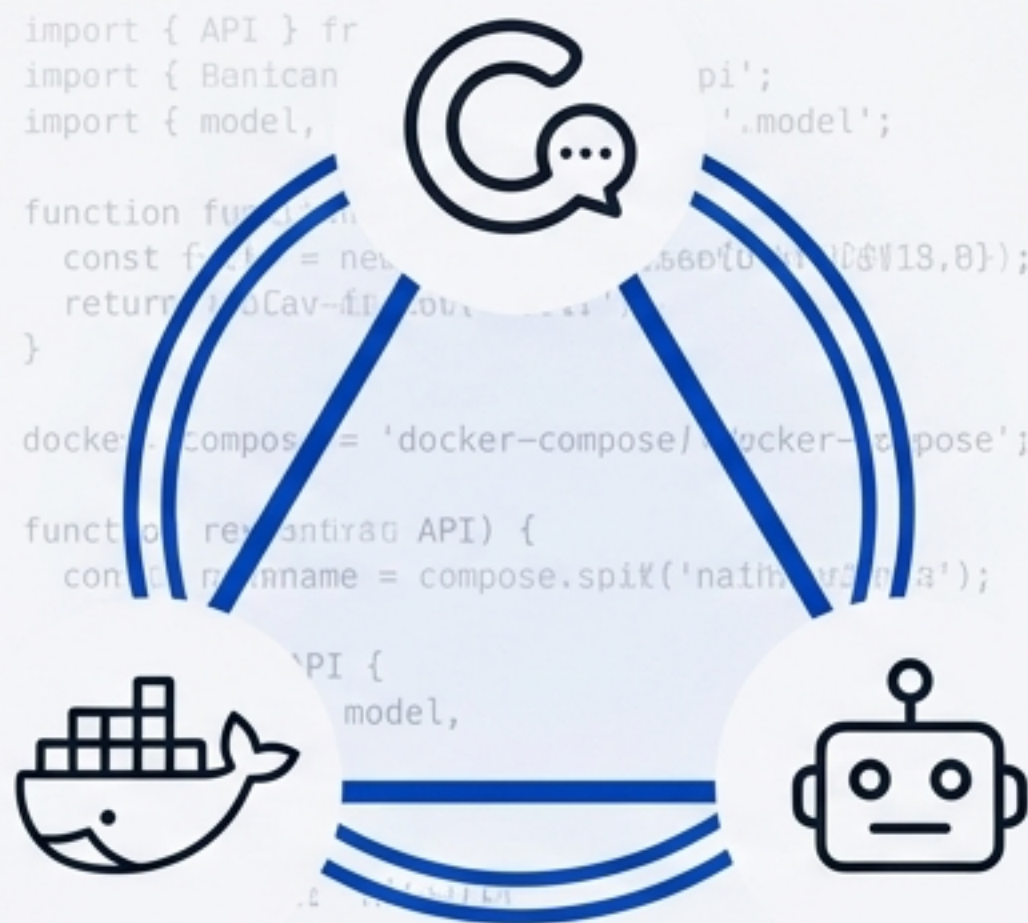


参考幻灯片 6: 核心配置

总结：你已成功构建私有化AI Bot开发平台

我们完成了：

-  **环境的搭建：**成功配置了支持容器化的开发环境。
-  **核心的连接：**将Coze平台与强大的云端大模型API打通。
-  **服务的部署：**使用行业标准工具 Docker Compose 实现了应用的自动化部署。



现在你可以：

-  在完全私有的环境中，安全地开发和调试AI应用。
-  自由集成和切换不同的模型服务提供商。
-  深入探索Coze的插件、工作流和知识库等高级功能。